

Liveness

Scribe: Jeffrey Chen

January 27, 2025

1 Introduction

In this chapter, we continue our discussion of fundamental properties in multi-robot navigation. Previously, we examined safety. Now, we shift our focus to liveness, a property that guarantees a robot continues making progress toward its goal. It is good practice to consider and identify these different properties for any navigation algorithm.

In dynamic robot navigation, a common challenge arises when a robot becomes stuck, unable to make progress toward its intended goal. When this occurs, we say that the robot is not live. Liveness, therefore, pertains to the ability of a robot to continuously advance toward its objective. Ensuring liveness is a fundamental requirement in navigation.

In this chapter, we examine the factors that can lead to liveness failures and explore established methodologies for maintaining liveness in multi-robot navigation systems.

2 Freezing Robot Problem

We begin by examining the causes of liveness breakdown, which is commonly known as the Freezing Robot Problem (FRP). The FRP occurs when a robot ceases to make progress toward its goal, despite being physically capable of movement. The formal condition for the FRP is given by the following equation:

$$|u|_{\Delta t}^r < \epsilon \wedge x_{t\epsilon\Delta t}^r \notin x_g \quad (1)$$

This equation consists of two conditions:

1. $|u|_{\Delta t}^r$ represents the magnitude of the robot's control term during some time duration Δt . If the robot is effectively not moving over this time duration, then $|u|_{\Delta t}^r$ is effectively zero.
2. $x_{t\epsilon\Delta t}^r \notin x_g$ states that the robot's position during this duration never reaches the goal state x_g .

If both conditions hold, the robot is considered frozen and unable to proceed toward its objective.

In many practical scenarios, heuristics and environmental factors are sufficient to jump start the robot and resolve the FRP. As a result, the FRP is often treated as a soft constraint. However, more critical challenges arise in the form of deadlocks, which can be considered a specific case of the FRP. We analyze deadlocks in both single-agent and multi-agent settings.

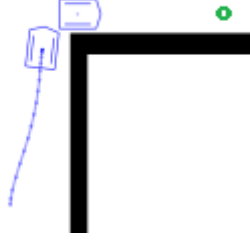


Figure 1: Single agent gets stuck.

3 Single-Agent Deadlock (DS-MPEPC)

We consider the scenario illustrated in Figure 1 where there is a stationary robot around the bend, and our robot encounters a deadlock when attempting to navigate past it toward a goal (denoted by the green circle) [1].

As always, it is helpful to consider the taxonomy of the problem. The taxonomy of this scenario is as follows:

1. Decentralized
2. Non-Communicative
3. Environment–Fully Observable, Agent–Partially Observable
4. Non-cooperative

In this setting, we have partial observability with respect to the agent because our robot lacks knowledge of the stationary robot’s position until it is in close proximity. If full agent observability were available, the robot could preemptively adjust its trajectory around the stationary obstacle. The deadlock arises because the robot only detects the stationary agent when it is already too close.

Additionally, this scenario is non-cooperative—if cooperation were possible, the robot could simply request the stationary robot to move, instantly resolving the deadlock.

A typical cost function for this navigation problem is given by:

$$J^i = \sum_{t=0}^T J_{\text{progress}} + J_{\text{action}} + J_{\text{collision}}$$

where:

- J_{progress} quantifies the extent of progress the robot is making toward its goal
- J_{progress} reflects the smoothness and continuity of the robot’s motion along its trajectory
- $J_{\text{collision}}$ accounts for collision avoidance and safety, ensuring the robot navigates without violating constraints such as CBF’s

Ideally, the optimal trajectory to resolve the deadlock should involve the robot taking a wider path around the stationary obstacle. However, in practice, the existing cost function fails to guide the robot toward the correct trajectory, ultimately resulting in deadlock.

To address this issue, the DS-MPEPC framework introduces a modified cost function by incorporating an additional terminal term. The purpose of this term is to incentivize the robot to select a trajectory that avoids deadlock with the stationary obstacle and instead navigates around it. The new cost function is as follows:

$$J^i = \sum_{t=0}^T J_{\text{progress}} + J_{\text{action}} + J_{\text{collision}} + J_{\text{terminal}} \quad (2)$$

where the terminal cost term is defined as:

$$J_{\text{terminal}} = -k[C_{\text{ttg}} \cdot C_{\text{ttc}}] \quad (3)$$

where constant $k > 0$, C_{ttg} represents the cost of time-to-goal, and is C_{ttc} represents the cost of time-to-collision.

C_{ttg} is defined such that:

- If a deadlock occurs and the time to reach the goal approaches infinity, then C_{ttg} tends to 1.
- If the goal remains reachable within a finite time, then C_{ttg} tends to 0.

Under deadlock conditions, C_{ttg} tends to 1, simplifying the terminal cost to:

$$J_{\text{terminal}} \simeq -k[C_{\text{ttc}}]$$

Minimizing this term encourages the robot to seek trajectories that maximize the time to collision, favoring a wider path around the stationary obstacle. This adjustment enables the robot to escape deadlock situations.

The key insight behind incorporating the terminal term is that in navigation scenarios where deadlocks do not occur, the terminal cost term is deactivated because C_{ttg} tends to 0. Thus, this additional terminal term only influences decision-making when deadlocks are present, ensuring a minimally invasive deadlock resolution mechanism.

We will explore the concept of minimally invasive resolution further in later sections.

4 Multi-Agent Deadlock

4.1 Social Mini Games

In a multi-agent setting, deadlocks often arise due to symmetry in the environment. When two agents are in a nearly symmetrical scenario—having similar distances to their respective goals and nearly equal velocities—they may arrive at a doorway simultaneously, as illustrated in Figure 2.

In this figure, the tips of the arrows indicate the starting positions of the robots. Both robots reach the doorway at the same time, with neither agent yielding, resulting in mutual obstruction. To resolve this, one robot must proceed first. But how is this decision made?

We refer to such scenarios as Social Mini-Games (SMG) [2].

Formally, recall our problem formulation tuple: $\langle k, x, U^i, T^i, J^i, O^i, \Omega^i, \tilde{\Gamma} \rangle$, recall $\tilde{\Gamma}$ represents the set of preferred trajectories that the agent would take in the absence of obstacles. If all $\tilde{\Gamma}^i, \tilde{\Gamma}^j$ are in conflict, then a Social Mini-Game occurs. Some common SMG scenarios are doorways, intersection, or hallways.

The formal connection between symmetry in the environment and multi-agent deadlock remains an open problem. However, we can intuitively observe the effects of symmetry:

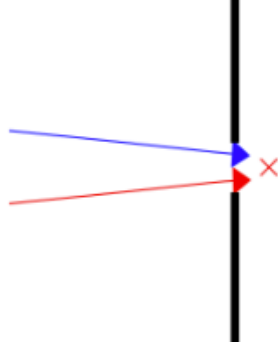


Figure 2: A Doorway Social Mini-Game.

- In asymmetrical scenarios, the agent with the greater distance naturally slows down or yields.
- In symmetrical scenarios, however, neither agent has incentive to let the other pass first, resulting in a deadlock

This challenge is particularly difficult when operating under the hardest taxonomy, where agents are non-communicative.

4.2 Fast, On-line Collision Avoidance for Dynamic Vehicles Using Buffered Voronoi Cells (Zhou et al., 2016)

We now examine an approach that aims to perform collision avoidance. Remember the goal is to achieve this for the most challenging setting, characterized by the following properties:

1. Decentralized
2. Non-Communicative
3. Environment–Partially Observable, Agent–Partially Observable
4. Non-cooperative

Previously, we studied double integrator dynamics, given by:

$$\begin{bmatrix} \dot{p}^i \\ \dot{v}^i \end{bmatrix} = \begin{bmatrix} 0 & I_{2 \times 2} \\ 0 & 0 \end{bmatrix} \begin{bmatrix} p^i \\ v^i \end{bmatrix} + \begin{bmatrix} 0 \\ I_{2 \times 2} \end{bmatrix} u^i$$

where p^i , v^i , and u^i represent the position, velocity, and control input (acceleration) of the i -th robot, respectively. The term $I_{2 \times 2}$ denotes the 2×2 identity matrix.

In contrast, this paper [7] utilizes single integrator dynamics:

$$\begin{bmatrix} \dot{p}_x \\ \dot{p}_y \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos \theta \\ \sin \theta \\ 0 \end{bmatrix} u_v + \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} u_w = \begin{bmatrix} \cos \theta & 0 \\ \sin \theta & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} u_v \\ u_w \end{bmatrix}$$

where p_x and p_y denote the robot's position in a two-dimensional plane, while θ represents its orientation with respect to a global reference frame. The control inputs are given by u_v and u_w , where u_v corresponds to linear velocity, and u_w represents angular velocity.

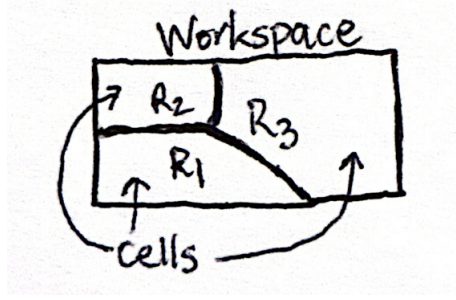


Figure 3: Voronoi Cells in a three robot workspace.

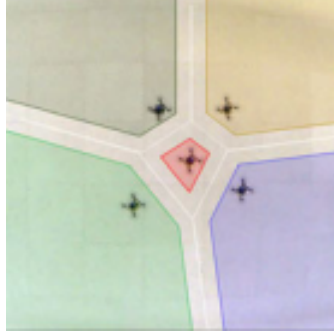


Figure 4: Buffered Voronoi Cells in a five robot workspace.

These dynamics indicate that the system follows a deterministic model, where the trajectory is fully determined by the control input. This relationship can be formally expressed as:

$$\Gamma = \Psi$$

Consider a workspace containing multiple robots. This paper introduces the concept of Voronoi Cells to partition the space dynamically. The process is as follows:

- Select an arbitrary point p from the workspace
- Compute the distance from p to each robot k : $p_d^k = |p - p^k| \forall k$
- Assign p to the nearest robot

An example of this partitioning for three robots, resulting in three Voronoi cells, is illustrated in Figure 3. Formally, the general Voronoi Cell of agent i is defined as:

$$\mathcal{V}_i = \left\{ \mathbf{p} \in \mathbb{R}^2 \mid \left(\mathbf{p} - \frac{\mathbf{p}_i + \mathbf{p}_j}{2} \right)^T \mathbf{p}_{ij} \leq 0, \forall j \neq i \right\}, \quad (4)$$

In order to account for robot size, we define a buffered Voronoi Cell, as seen in Figure 4:

$$\bar{\mathcal{V}}_i = \left\{ \mathbf{p} \in \mathbb{R}^2 \mid \left(\mathbf{p} - \frac{\mathbf{p}_i + \mathbf{p}_j}{2} \right)^T \mathbf{p}_{ij} + r_s \|\mathbf{p}_{ij}\| \leq 0, \forall j \neq i \right\}. \quad (5)$$

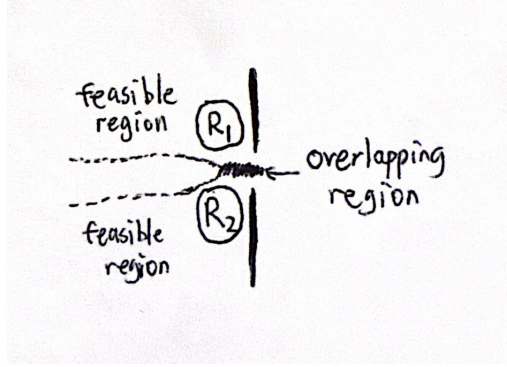


Figure 5: Voronoi cells overlapping, leading to deadlock.

where r_s represents robot size.

By restricting agents to move only within and along their respective Voronoi Cells, the approach inherently guarantees safety, as no two agents can occupy the same space.

However, when revisiting the doorway SMG scenario, we encounter a fundamental issue with this Voronoi partitioning. As two robots approach the doorway, they end up stopping because there is no Voronoi cell that is not overlapping at the doorway, as illustrated in Figure 5.

We will explore how the deadlock is solved later.

4.3 Safety barrier certificates for heterogeneous multi-robot systems (Wang et al. 2017)

Revisiting the Control Barrier Functions (CBF) framework [6] from the Safety chapter, we can observe a direction connection with the deadlocks in Zhou et al.

Recall that the CBF constraint is defined as:

$$\dot{h}(x) \geq -\alpha h(x)$$

The CBF function that we derived is:

$$h(x) = \|\bar{p}\| - \frac{\bar{v}_r^2 r}{2(\alpha_i + \alpha_j)} - d \geq 0 \quad (6)$$

$\bar{v}_r$ represents the component of the relative velocity along $\bar{p}$.

Now, consider a scenario where we increase the symmetry between two robots, gradually forming a Social Mini-Game (SMG). Referring to Figure 6, we observe that as symmetry increases, the $\bar{v}_r$ also increases.

Examining Equation 6, we see that as $\bar{v}_r$ increases, $h(x)$ decreases. Thus, as symmetry approaches perfect symmetry, the robot tends toward unsafe regions, where no feasible, safe actions exist for the robots. This results in a deadlock.

5 Deadlock Resolution: Right Hand Rule

Deadlock resolution requires breaking symmetry by perturbing the robots' behavior. In this section, we examine a heuristic known as the Right-Hand Rule, where the rightmost agent moves first. The

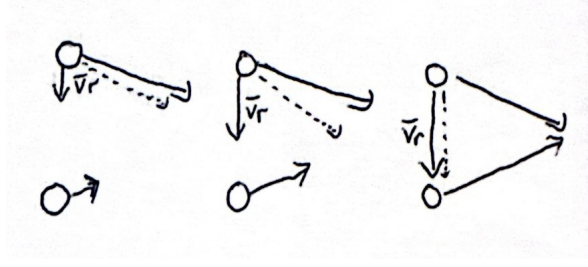


Figure 6: As symmetry increases, $\bar{v}_r$ increases.

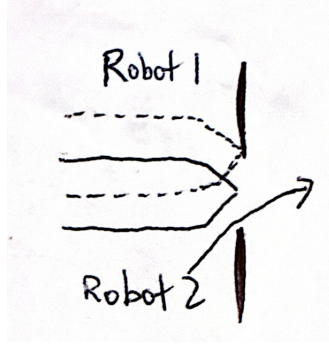


Figure 7: Shifted Voronoi cells allow Robot 2 to proceed first.

fundamental idea is to induce agents to move left, ensuring the rightmost agent has the right of way. Note that implementing this rule eliminates the possibility of a non-cooperative approach.

5.1 Zhou et.al.

Revisiting the Voronoi-based deadlock scenario, we explore how the Right-Hand Rule is implemented. The key modification involves adjusting the point assignment formula to shift Voronoi boundaries leftward relative to their original positions, as illustrated in Figure 7.

With this modified assignment, the right robot proceeds first.

5.2 Wang et.al.

Returning to the CBF framework, how can we enforce the Right-Hand Rule? The solution lies in breaking the CBF constraint into two separate terms, corresponding to the left and right regions around the robot. This results in the following condition:

$$\dot{h}(x) \geq -k_L \alpha h(x) \wedge -k_R \alpha h(x) \quad (7)$$

where k_L and k_R represent scaling coefficients for the left and right regions, respectively. These coefficients determine how the robot perceives safety on each side.

To bias motion to the left, we increase k_L and decrease k_R . This adjustment causes the robots to perceive their right sides as more unsafe than the left, which induces the robots to move left, ensuring compliance with the Right-Hand-Rule.



Figure 8: Agents follow a RL-based right hand rule.

5.3 Socially Aware Collision Avoidance with Deep Reinforcement Learning (SACADRL)

Finally, we examine a Reinforcement Learning based approach that incorporates a strategy similar to the CBF-based method [3].

This method leverages a doorway Social Mini-Game (SMG) scenario, as depicted in Figure 8. Similarly to Wang et al., this approach divides the robot’s left and right regions. However, rather than modifying constraints, the reward function is structured as follows:

- Higher rewards for movements to the right
- Lower rewards for movements to the left

This leads to behavior that adheres to the Right-Hand-Rule.

6 Autonomous Intersection Management

General approaches rely on a central node that handles priority assignments, determining which vehicle moves first based on predefined rules. In such systems, the highest-priority agent proceeds before others. This approach can be considered minimally invasive, as vehicles do not need to alter their trajectories. However, to achieve this, the taxonomy is relaxed to:

1. Centralized or Distributed
2. Co-operative
3. Communicative
4. Partially Observable

Some broad categories of autonomous intersection management include:

- Scheduling methods
- Reservation systems
- Auction-based systems

These approaches fundamentally rely on a centralized oracle-based strategy, similar to a traffic light system that dictates the flow of traffic.

Autonomous intersection management differ from heuristic methods such as right-hand rule navigation. While such heuristics still reflect elements of centralized control, they less restrictive than fully centralized intersection management systems, which directly dictate traffic flow.

7 Toward Minimally Invasive Deadlock Prevention

In exploring multi-robot navigation, a crucial distinction arises between deadlock resolution and deadlock prevention. Approaches in the current literature focus on deadlock resolution—intervening after the deadlock has already occurred. In practice, this often means a cluster of robots converging at a bottleneck (such as a doorway), with resolution methods orchestrating which robots proceed first.

True deadlock prevention, however, remains an open problem. This would involve robots preemptively detecting an impending deadlock and adjusting their movement to avoid it altogether, leading to smoother and more agile motion.

When considering solutions, we aim to work towards the following:

- Decentralized vs. Centralized: We prioritize decentralized approaches that do not rely on a central controller.
- Non-Communicative vs. Communicative: We assume robots cannot communicate with one another.
- Non-Cooperative vs. Cooperative: Robots act in their own self-interest.
- Minimally-invasive vs Invasive: Our goal is to develop minimally invasive methods. Current strategies involve perturbing the robots’ trajectories. Minimally invasive approaches would only adjust a robot’s speed rather than altering its trajectory entirely.

An example of a minimally invasive approach is DS-MPEPC, which has demonstrated success in single-robot navigation. However, multi-robot settings present a greater challenge. This leads to the following open research question:

How can we prevent multi-robot deadlocks in a minimally invasive manner, within a fully decentralized, non-cooperative, partially observable, and non-communicative setting?

While current methodologies are effective to a certain extent, they often rely on artificial or invasive interventions. In contrast, humans navigate seamlessly without requiring such techniques. This motivates further research toward developing human-like navigation strategies that minimize disruption while ensuring liveness.

7.1 Liveness Sets

A promising concept in the pursuit of minimally invasive deadlock resolution is liveness sets. These can be thought of as an analog to safe sets, with the distinction that liveness sets are defined in terms of joint speeds.

Definition 1. At any time t , given a configuration of k robots, a *liveness set* is defined as a union of convex sets, $C_\ell(t) \subseteq \mathbb{R}^k$, of joint velocity $v_t = (v_t^1, v_t^2, \dots, v_t^k)^\top$, such that:

$$v_t^i - \zeta v_t^j \geq 0, \quad \forall i \neq j, \quad \zeta \geq 2.$$

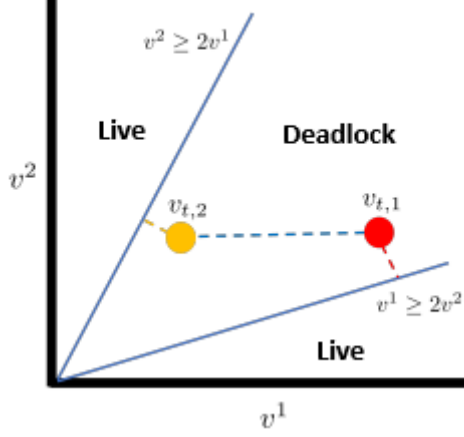


Figure 9: Liveness Set for 2 Robot Scenario.

For a two-robot scenario, the following inequalities define the liveness constraints:

$$\begin{aligned} v_t^1 &\geq \zeta v_t^2 \\ v_t^2 &\geq \zeta v_t^1 \end{aligned}$$

which can be rewritten as:

$$\begin{aligned} v_t^1 - \zeta v_t^2 &\geq 0 \\ v_t^2 - \zeta v_t^1 &\geq 0 \end{aligned}$$

This can be compactly expressed in matrix form as:

$$Av \geq 0$$

where

$$A = \begin{bmatrix} 1 & -\zeta \\ -\zeta & 1 \end{bmatrix}$$

and

$$v = \begin{bmatrix} v_1 \\ v_2 \end{bmatrix}.$$

By casting $Av \geq 0$ as a Control Barrier Function (CBF) constraint, we can formally integrate liveness sets into robot dynamics, ensuring that robots maintain motion in a minimally invasive manner.

Figure 9 shows an example with $\zeta = 2$. We see that both robots are currently inside the deadlock zone. The red dotted line and the yellow dotted line represent the minimally invasive perturbations for the two robots to return to liveness sets, which can be thought of as the section to the left and right of the deadlock zone.

8 Follow-Up Exercises

1. Read *Why Does Symmetry Cause Deadlocks?* [5] and establish symmetry/SMG equivalence.
2. Explore how to use Reachable Bezier Polytypes [4] for Liveness.

References

- [1] Senthil Hariharan Arul, Jong Jin Park, and Dinesh Manocha. Ds-mpepc: Safe and deadlock-avoiding robot navigation in cluttered dynamic scenes. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2256–2263, 2023.
- [2] Rohan Chandra, Vrushabh Zinage, Efstathios Bakolas, Peter Stone, and Joydeep Biswas. Deadlock-free, safe, and decentralized multi-robot navigation in social mini-games via discrete-time control barrier functions. 2024.
- [3] Yu Fan Chen, Michael Everett, Miao Liu, and Jonathan How. Socially aware motion planning with deep reinforcement learning. 03 2017.
- [4] Noel Csomay-Shanklin and Aaron D. Ames. Bezier reachable polytopes: Efficient certificates for robust motion planning with layered architectures, 2024.
- [5] Jaskaran Grover, Changliu Liu, and Katia Sycara. Why does symmetry cause deadlocks? *IFAC-PapersOnLine*, 53(2):9746–9753, 2020. 21st IFAC World Congress.
- [6] Li Wang, Aaron Ames, and Magnus Egerstedt. Safety barrier certificates for heterogeneous multi-robot systems. In *2016 American Control Conference (ACC)*, pages 5213–5218, 2016.
- [7] Dingjiang Zhou, Zijian Wang, Saptarshi Bandyopadhyay, and Mac Schwager. Fast, on-line collision avoidance for dynamic vehicles using buffered voronoi cells. *IEEE Robotics and Automation Letters*, 2(2):1047–1054, 2017.